

Homework 2: Dynamic Programming 1D

Guideline

Homework chatbot. [Open the homework Gem here](#). Use this Gem when working through the AI-related sections. The bot is already curated with the right answers and hints to provide the right level of guidance to help you learn.

Showing your honest attempt can only help. There is no advantage in cheating. I give 100% credit on attempts, even if they are incorrect. Cheating on homework will also only hurt you. If you cheat, I won't be able to give you the proper feedback to help you learn, nor adjust the difficulty of the quizzes. You will also likely fail on the in-person mastery quizzes, which test concepts tackled in the homework.

Collaboration: Only after solo attempt. You may collaborate and use external resources, but (1) you are encouraged to do the homework **alone first** and (2) you must **attribute** these external resources when you do use them.

AI: Follow guideline. The problems will be very clear on when you are allowed to use AI. Please follow them. Here is why: (1) [Unregulated usage of AI harms learning](#) (2) Even in the age of AI, your [AI-independent ability matters](#). The **only** AI tool you should use on this homework is the homework-specific [Gemini Gem](#) that I have created for you.

The 4-step cookbook. Every problem below uses the same recipe from lecture:

1. **Define subproblems.** `dp[i] = "best" way to handle some prefix or suffix of the input`
2. **Find recurrence.** `dp[i] = f(dp[i-1], dp[i-2], ...)`
3. **Specify order.** `i = 0->n-1` or `i = n-1->0` ?
4. **Base case.** `dp[0] = ?`

Each problem asks you for steps 1 and 2 explicitly. Steps 3 and 4 you will have to sort out yourself when you write the code -- that is deliberate.

Submission. Upload all files to the Homework 2 assignment on Canvas. **5 files total:**

1. **One answer document** containing your answers to every question below. Name it `lastname-hw2.pdf` (or `.txt`). LaTeX isn't required -- plain text, Google Docs → Download as PDF, or Word → Save as PDF are all fine.
2. **Four AI chat transcript PDFs**, one for each AI-check step. Start a **new chat** in the Gem for each step, so that each transcript covers only that step:
 - `brainpower-b.pdf`, `brainpower-f.pdf`
 - `wordbreak-b.pdf`, `wordbreak-f.pdf`

Accepted file formats: `.pdf` for everything; `.txt` is also fine for the answer document. Do **not** paste chat content into your answer document, and do **not** submit a link to a Google Doc -- upload the file itself.

Due: Sunday, October 4, 2026 at 11:59pm. See the syllabus for the late-day policy.

Points: 39 total. Remember, I give full points to every question that was attempted, and 0 only for a question left blank. What matters is that you have **thought** through the problems enough to prepare yourself for the quizzes.

1. House robber recap (7 pts total)

Do **not** use AI for this section. Refer to the lecture materials first, then close them and write from memory. This is the reconstruction exercise -- looking up the answer while you write defeats the purpose.

You are robbing houses along a street. Each house has some money. You **cannot rob two adjacent houses**, or the police get called. Return the maximum money you can rob.

Our running example from lecture:

House index	0	1	2	3
money	8	9	8	3

The answer is 16 -- rob houses 0 and 2.

[Leetcode link](#)

(i) Define the subproblem (1 pt)

Write down, in **one line**, what `dp[i]` means. Be precise about which houses `dp[i]` considers.

(ii) Find the recurrence (1 pt)

Express `dp[i]` in terms of smaller subproblems. Show the exhaustive cases you are taking the max over.

(iii) Compute the `dp[]` array by hand (1 pt)

Using **your** definition, compute `dp[0]`, `dp[1]`, `dp[2]`, `dp[3]` for the example above. Show the arithmetic for each cell, not just the final number.

(iv) Why is the recurrence correct? (1 pt)

Write between 2 and 5 sentences. Imagine the **optimal** robber. Argue that their plan must fall into one of your cases, and that within each case they must have behaved optimally on the subproblem.

(v) Runtime (1 pt)

What is the big-O runtime, and why?

(vi) Space complexity (1 pt)

What is the big-O space usage? Can it be reduced? If so, to what, and what makes that possible?

(vii) Code (1 pt)

Now code it up on leetcode: [House Robber](#). Still no AI -- if you got the recurrence right above, the code follows directly from it.

Turn in your final code, pasted into your answer document. Also paste your Leetcode submission URL showing a successful submission.

2. Max subarray sum recap (8 pts total)

Do **not** use AI for this section. Same rules as problem 1 -- from memory.

Given an integer array `nums`, find the contiguous subarray with the largest sum and return that sum.

You have seen this problem before. In Homework 1 you solved it with a divide-and-conquer approach in $O(N \log N)$, and I promised we would come back and do it in $O(N)$. This is that.

Our running example from lecture:

```
nums = [-2, 1, -3, 4, -1, 2, 1, -5, 4]
```

The answer is `6` -- the subarray `[4, -1, 2, 1]`.

[Leetcode link](#)

(i) Define the subproblem (1 pt)

Write down, in **one line**, what `dp[i]` means. Note that the "obvious" definition does not work here -- your definition needs an extra condition on it. State that condition clearly.

(ii) Find the recurrence (1 pt)

Express `dp[i]` in terms of smaller subproblems. Show the exhaustive cases you are taking the max over.

(iii) Read the answer off the `dp[]` array (1 pt)

Once the whole `dp[]` array is filled in, **how do you get the final answer out of it?** This is different from house robber, where the answer was simply the last cell. Explain in 1 to 2 sentences why the extra condition in your definition forces this difference.

(iv) Compute the `dp[]` array by hand (1 pt)

Using **your** definition, compute `dp[0]`, `dp[1]`, `dp[2]`, `dp[3]` for the example above. Show the arithmetic for each cell.

(v) Why is the recurrence correct? (1 pt)

Write between 2 and 5 sentences arguing why your recurrence is correct.

(vi) Runtime (1 pt)

What is the big-O runtime, and why?

(vii) Space complexity (1 pt)

What is the big-O space usage? Can it be reduced? If so, to what, and what makes that possible?

(viii) Code (1 pt)

Now code it up on leetcode: [Maximum Subarray](#). Still no AI. Note that leetcode will accept your HW1 divide-and-conquer solution too -- submit the **O(N) dynamic programming** one.

Turn in your final code, pasted into your answer document. Also paste your Leetcode submission URL showing a successful submission.

3. Questions and brainpower (12 pts total)

You are given the questions of an exam. You must process them **in order**, deciding for each whether to **solve** or **skip** it. Solving question i earns you `points[i]`, but you are then **unable to solve the next `brainpower[i]` questions**. Skipping question i costs nothing and you move on to question $i+1$. Return the maximum points you can earn.

Our running example:

Question ID	0	1	2	3
points	3	4	4	2
brainpower	2	3	4	5

The answer is `5` -- solve question 0 (earning 3, which blocks questions 1 and 2), then solve question 3 (earning 2).

[Leetcode link](#)

(a) No-AI-attempt planning

Understand the problem first, then start a stop watch for **30 minutes** and tackle this on your own. Use the full 30 minutes. If you give up before you have a complete answer, that's fine -- an incomplete attempt still earns full credit.

(i) The natural first attempt, and why it fails (1 pt)

Start with the definition that worked for house robber:

```
dp[i] = max points if we consider questions 0..i
```

Try to write a recurrence for it. You will get stuck. Give an **intuitive description**, in 2 to 4 sentences, of *why* you get stuck. What information does `dp[i-1]` fail to tell you?

(ii) A definition that works (1 pt)

Now find a definition that does work. **Hint:** what if you filled the `dp[]` array from **right to left** instead of left to right?

Write down, in one line, what your new `dp[i]` means.

(iii) Find the recurrence (1 pt)

Express `dp[i]` in terms of subproblems, showing the exhaustive cases. You do not have to worry about index out of bounds.

(b) AI plan check

Use the homework chatbot to check your answers from part (a). Here are some example prompts:

- **Verifying.** "Here is the problem statement (copy paste). Here is my dp definition and recurrence (copy paste). Are these right?"
- **Brainstorming.** "I'm stuck. Can you give me a hint?"
- **Asking for clarification.** "Why doesn't the left-to-right definition work here?"

(i) Chat transcript (1 pt)

Turn in a **chat transcript** as a PDF. Name it `brainpower-b.pdf`. Do **not** copy paste the chat content into your answer document.

(ii) Human reconciliation (1 pt)

Summarize in your answer document. Any corrections you made? Any new learnings? You can also say "Nope, I got everything right."

(c) Compute the `dp[]` array by hand (1 pt)

Now that your definition has been checked, use it to compute `dp[0]`, `dp[1]`, `dp[2]`, `dp[3]` for the example above. Show the arithmetic for each cell, not just the final number.

(d) Why is the recurrence correct? (1 pt)

Write between 2 and 5 sentences. Imagine the **optimal** test-taker. Argue that their plan must fall into one of your exhaustive cases, and that within each case they must have behaved optimally on the subproblem.

(e) Code (1 pt)

Start a stop watch for **30 minutes** and code this on leetcode without using AI. Imagine that this is you having to write code on the whiteboard for an interview, and you are under time pressure.

If you have exceeded 30 minutes, you **may** use the homework AI to help you succeed and get a successful leetcode submission.

Turn in your final code, pasted into your answer document. Also paste your Leetcode submission URL showing a successful submission.

(f) AI code check

Copy paste your code to the [homework gem](#) and ask for AI review. Here is an example prompt: "Here is the problem statement (copy) and here is my code (copy). Q1. Can you evaluate for correctness? Q2. Can you help improve my code style?"

(i) Chat transcript (1 pt)

Turn in the chat transcript PDF. Name it `brainpower-f.pdf`. Do **not** copy paste the chat content into your answer document.

(ii) Human reconciliation (1 pt)

Summarize in your answer document. Any corrections you made? Any new learnings? You can also say "Nope, I got everything right."

(g) Runtime (1 pt)

What is the big-O runtime of your solution, and why?

(h) Space complexity (1 pt)

What is the big-O space usage? Can it be reduced the way it could for house robber? Why or why not?

4. Word break (12 pts total)

Given a string `s` and a dictionary of strings `wordDict`, return `true` if `s` can be segmented into a space-separated sequence of one or more dictionary words. You may reuse a dictionary word as many times as you like.

Heads up: this problem breaks the mold in two ways. First, `dp[i]` here is a **boolean**, not a "best" value -- not every dynamic program maximizes something, some just answer "is this possible?". Second, the recurrence will need an **inner loop**, unlike every recurrence in lecture where `dp[i]` looked back a fixed number of cells.

Our running example:

```
s = "hime", wordDict = ["h", "hi", "him", "me"]
```

The answer is `true` -- "hime" splits into "hi" + "me".

A second example, to show the answer is not always yes:

```
s = "hime", wordDict = ["hi", "him"]
```

The answer is `false` -- taking "hi" leaves "me" behind, and taking "him" leaves "e" behind. Neither leftover is in the dictionary.

[Leetcode link](#)

(a) No-AI-attempt planning

Understand the problem first, then start a stop watch for **30 minutes** and tackle this on your own. Use the full 30 minutes. If you give up before you have a complete answer, that's fine -- an incomplete attempt still earns full credit.

(i) Define the subproblem (1 pt)

Write down, in **one line**, what `dp[i]` means. Since `dp[i]` is a boolean, your one-liner should read like "`dp[i]` is true when ...".

(ii) Find the recurrence (1 pt)

Express `dp[i]` in terms of smaller subproblems. Since the last word in a segmentation can be any length, your cases are no longer a simple "take it or leave it" -- you have to consider **every possible split point**. Write the recurrence so that this is explicit.

(b) AI plan check

Use the homework chatbot to check your answers from part (a). Here are some example prompts:

- **Verifying.** "Here is the problem statement (copy paste). Here is my dp definition and recurrence (copy paste). Are these right?"
- **Brainstorming.** "I'm stuck. Can you give me a hint?"
- **Asking for clarification.** "Why does this recurrence need an inner loop when house robber didn't?"

(i) Chat transcript (1 pt)

Turn in a **chat transcript** as a PDF. Name it `wordbreak-b.pdf`. Do **not** copy paste the chat content into your answer document.

(ii) Human reconciliation (1 pt)

Summarize in your answer document. Any corrections you made? Any new learnings? You can also say "Nope, I got everything right."

(c) Compute the `dp[]` array by hand (1 pt)

Using your checked definition, compute `dp[0]`, `dp[1]`, `dp[2]`, `dp[3]` for the **first** example (`s = "hime"`, `wordDict = ["h", "hi", "him", "me"]`). These are booleans, so for each cell state true or false **and** name the split that justifies it.

(d) Why is the recurrence correct? (1 pt)

Write between 2 and 5 sentences. Imagine a valid segmentation of the string. Argue what needs to happen for the last word, and the remaining prefix of the string.

(e) Code (1 pt)

Start a stop watch for **30 minutes** and code this on leetcode without using AI. Imagine that this is you having to write code on the whiteboard for an interview, and you are under time pressure.

If you have exceeded 30 minutes, you **may** use the homework AI to help you succeed and get a successful leetcode submission.

Turn in your final code, pasted into your answer document. Also paste your Leetcode submission URL showing a successful submission.

(f) AI code check

Copy paste your code to the [homework gem](#) and ask for AI review. Here is an example prompt: "Here is the problem statement (copy) and here is my code (copy). Q1. Can you evaluate for correctness? Q2. Can you help improve my code style?"

(i) Chat transcript (1 pt)

Turn in the chat transcript PDF. Name it `wordbreak-f.pdf`. Do **not** copy paste the chat content into your answer document.

(ii) Human reconciliation (1 pt)

Summarize in your answer document. Any corrections you made? Any new learnings? You can also say "Nope, I got everything right."

(g) Runtime (1 pt)

What is the big-O runtime of your solution, and why? Be careful here -- comparing two strings is not free. Count the cost of the substring work too, not just the loops.

(h) Space complexity (1 pt)

What is the big-O space usage?

(i) Optimizing the inner loop (1 pt)

Your inner loop probably checks **every** possible split point. Does it have to?

Describe at least one optimization that lets you skip work, and say whether it actually improves the big-O or only makes things faster in practice. **Hint:** think about the **longest word** in the dictionary. Also think about what you can do the moment you have found one valid split for `dp[i]`.